

경남대 RISE 피지컬AI 사관학교 — 8월 Agentic AI 특강 상세교안(안) v2

- 총 시수: 4일 × 4시간 = 16시간
- 대상: 사관학교 모집 학생 39명(컴퓨터공학 계열 37명, C/C++ 학습 경험, 파이썬 전환 가능)
- 목적: 본 과정(9월 개시) 이탈 방지를 위한 임팩트형 특강 — "다른 곳에서 배울 수 없는 과정"임을 4일 안에 체감시킴
- 기반 자산: 호서대 RISE 과정 교안(에이전틱 디자인 패턴 영상, SDD/OpenSpec 블록, Supabase 바이브코딩, MCP·멀티에이전트 재설계 구조)을 **피지컬 AI 도메인으로 전면 치환**하여 재구성

1. 설계 원칙

1. **도메인은 처음부터 끝까지 피지컬 AI** — 관리시스템(WMS류)이 아니라 「인지(Perception) → 판단(Decision) → 행동(Actuation)」의 물리 페루프가 과정 전체의 뼈대다. 학생이 만든 에이전트가 마지막에는 (시뮬레이션된) 설비·로봇에 **실제 명령을 내린다**.
2. **매일 1개의 '와' 순간으로 마감** — 각 일자 마지막 30분은 반드시 동작하는 결과물 데모로 끝낸다. 특강의 KPI는 지식 전달량이 아니라 본 과정 등록 유지율이다.
3. **4비트 학습 루프 적용** — 모든 랩은 「작은 성공 → 의도된 불편 → 대조 해결 → 자산화」 순서로 설계하고, Day 1의 '의도된 불편'이 Day 2 SDD의 동기가 되도록 일자 간을 연결한다.
4. **실습 비중 60~70%, 모든 랩 서두에 '왜'** — 호서대 운영 교훈(실습 90% + 가이드 과다 → 피로·이탈)을 반영, 핸드온 가이드는 일자당 30페이지 이내, 각 랩 첫 반 페이지에 "이 랩을 왜 하는가"를 명시한다.
5. **کم공생 눈높이 조정** — 이상감지 알고리즘 등 핵심 로직은 라이브러리 호출이 아닌 직접 구현 구간을 둔다(전공 자부심 자극 = 임팩트 요소).

2. 스루라인 시나리오: "K-정밀 스마트팩토리 관제 페루프"

가상의 경남(창원) 소재 정밀기계 부품 제조사 **K-정밀**. CNC 설비 6대와 자율이동로봇(AMR) 2대가 있는 공장을 **강사측 제공 팩토리 시뮬레이터**(3절) 위에서 운영한다. 4일간 학생은 이 공장의 인지→판단→행동 페루프를 단계별로 완성한다.

일자	페루프 단계	학생이 만드는 것	누적 결과물
Day 1	인지 (보기)	설비·로봇 실시간 관제 대시보드(바이브코딩)	관제화면 v0
Day 2	인지의 구조화	스펙 기반 관제 시스템: 설비 마스터·작업지시·알람 규칙	+ DB 백엔드
Day 3	판단 (알기)	센서 이상감지 알고리즘 + 진단 에이전트(MCP)	+ 이상감지·진단
Day 4	행동 (움직이기)	멀티에이전트 페루프: 감지→진단→ 설비 제어·로봇 파견 (HITL 승인)	완성된 페루프

"보기 → 알기 → 움직이기"의 3단 서사로, 마지막 날 자기 에이전트가 이상 설비를 감속시키고 정비 로봇을 보내는 장면이 특강 전체의 클라이맥스다. 지역 제조업 맥락을 쓰는 이유: 사업 참여기업(15개사)·채용약정(4개사)과의 연결 서사, "취업으로 이어지는 기술" 메시지가 본 과정 유지의 가장 강한 동기이기 때문.

3.팩토리 시뮬레이터 (강사측 사전 제작 — 핵심 준비물)

하드웨어 없이 피지컬 AI 페루프를 성립시키는 장치. 웹 기반 2D 공장 뷰 + API 서버로 구성한다.

- 상태: CNC 6대(온도·진동·rpm·가동상태), AMR 2대(위치·배터리·적재상태)가 1초 주기로 변동, Supabase에 스트리밍 적재
- 제어 API(= Day 4에서 MCP 도구로 노출): `set_equipment_speed(id, rpm)`, `stop_equipment(id)`, `dispatch_robot(robot_id, target)`, `ack_alarm(id)`
- 시나리오 주입: 강사 콘솔에서 이상 이벤트(온도 드리프트, 진동 스파이크, 센서 결측) 발생 시 각 제어 가능
- 학생별 인스턴스 분리(팀/개인 네임스페이스) — 서로의 제어 명령이 충돌하지 않도록
- 폴백 경로: 시뮬레이터 제작이 일정상 어려우면, Supabase 상태 테이블 + 상태 갱신 스크립트로 대체하고 2D 뷰 대신 대시보드 갱신으로 시연(임팩트는 줄지만 과정 성립에는 문제 없음)

4.공통 실습 환경

- IDE: VS Code + Claude 플러그인 (버전 안정성 기준 확정 조합)
- DB: Supabase (가입·프로젝트 생성 자체를 Day 2 실습 항목에 포함)
- 저장소: GitHub (스펙 깃데기 배포·빈칸 채우기 교수법용)
- 언어: Python (C/C++ 경험 전제, Day 1에 30분 전환 브릿지 제공)
- 계정: 39명 규모이므로 자사 서버/일괄 발급 방식 우선 검토 (미확정)

5.사전 준비 및 리스크

항목	내용	기한
팩토리 시뮬레이터	3절 사양 제작 + 39명 동시 부하 테스트, 폴백 경로 병행 준비	특강 2주 전
실습 계정	39명 × Claude 사용량 — 자사 서버 경유 vs 개별 구독 결정	특강 2주 전
ProcessGPT 포지셔닝	딥에이전트 대기시간 이슈로 특강에서는 강사 데모만, 핸즈온은 본 과정으로 이연	Day 4 리허설
센서 데이터셋	이상 구간을 심은 합성 시계열(부록 A) — 시뮬레이터 로그와 동일 스키마	특강 1주 전
네트워크	강의장 동시 39명 Supabase/API/시뮬레이터 호출 부하 테스트	특강 1주 전
가이드 분량	일자당 핸즈온 가이드 30p 이내 검수	교안 완성 시

Day 1 — 보기: 코드를 짜는 시대에서, 시스템을 말하는 시대로

학습목표

- 1. 피지컬 AI를 인지→판단→행동 페루프로 정의하고, 에이전틱 AI가 그 '판단·행동'을 맡는 구조를 설명할 수 있다.
- 2. 5대 에이전틱 디자인 패턴의 이름과 용도를 구분할 수 있다.
- 3. 프롬프트만으로 시뮬레이터와 연결된 실시간 관제 대시보드를 생성할 수 있다.
- 4. 프롬프트 단독 개발의 한계(컨텍스트 붕괴)를 체험으로 안다. ← Day 2 동기

시간표 (240분)

시간	블록	방식
00:00-00:15	오리엔테이션: 4일의 지도 — "보기·알기·움직이기"	강의
00:15-01:10	강의 1: 피지컬 AI × 에이전틱 AI	강의+영상
01:10-01:40	셋업 랩: 개발환경 + 파이썬 브릿지	실습
01:40-01:50	휴식	
01:50-03:40	Lab 1-1: 바이브코딩 — K-정밀 관제 대시보드	실습
03:40-04:00	랩업: 데모 쇼케이스 + 무너진 경험 회수	발표

강의 1 슬라이드 목차 (약 24장) 1-4. 피지컬 AI란 무엇인가: 인지(센서)→판단(AI)→행동(액추에이터) 페루프 — 자율주행·휴머노이드·스마트팩토리가 전부 이 구조다 5-7. 왜 지금인가: LLM이 '판단'의 범용 엔진이 되면서 페루프의 병목이 풀렸다 + 채용시장·참여기업이 원하는 인재상 8-10. 에이전트 = 뇌 + 도구 + 루프 + 자율성 — 챗봇·코파일럿과의 차이 11-17. 에이전틱 디자인 패턴 5종 (호서대 영상 재활용, 각 1분 30초): Prompt Chaining / Routing / Parallelization / Orchestrator-Worker / Evaluator-Optimizer — 각 패턴을 공장 사례로 리프레이밍(예: Routing = 알람 등급별 대응 분기) 18-20. 오늘의 무대 소개: K-정밀 팩토리 시뮬레이터 투어(강사 시연) — "4일 뒤 여러분의 에이전트가 이 공장을 움직인다" 21-24. 오늘 실습 예고: "오늘은 이 공장을 '보는' 눈을 만든다 — 코드 한 줄 안 치고"

셋업 랩 (30분) — 핸드온 가이드 §1

- VS Code + Claude 플러그인 설치 확인(사전 안내문 발송 전제, 현장은 트러블슈팅 위주), GitHub 로그인·실습 저장소 클론
- 파이썬 브릿지 1페이지: C ↔ Python 대응표(포인터→참조, for·함수·클래스 대응) — "여러분은 이미 더 어려운 걸 배웠다"
- 시뮬레이터 접속 확인: 자기 네임스페이스의 공장 화면 열기

Lab 1-1: 바이브코딩 — 관제 대시보드 (110분) — 핸드온 가이드 §2

왜 하는가: 피지컬 AI의 첫 단계는 물리 세계를 '보는' 것이다. AI에게 말로 시키는 개발이 어디까지 되는지, 어디서부터 안 되는지도 함께 확인한다 — 안 되는 지점이 내일 배울 내용이다.

- Step 1 (작은 성공, 40분): 제공 프롬프트 시드로 시뮬레이터 API를 읽어 CNC 6대·AMR 2대의 상태(온도·진동·가동률·로봇 위치)를 보여주는 실시간 대시보드 생성 → 브라우저에서 자기 공장이 움직이는 걸 확인. 이후 자유 프롬프트로 커스터마이즈 1건.
- Step 2 (의도된 불편, 40분): 요구사항 카드 5장 순차 추가("온도 임계 알람 표시" → "설비별 이력 그래프" → "AMR 경로 표시" → "알람 담당자 배정" → "야간모드/권한 분리"). 3~4번째에서 기존 기능이 깨지는 경험 유도.
- Step 3 (대조 예고, 20분): 강사가 동일 요구를 스펙 문서 기반으로 한 번에 지시하는 시연 → 무너지지 않음. "차이가 뭘까?" 질문만 던지고 답은 내일로.
- Step 4 (자산화, 10분): 저장소 커밋 + 프롬프트 로그 저장.

랩업 (20분): 지원자 2~3명 대시보드 공유, 창의 커스터마이즈 즉석 투표. 마지막 1장: "오늘 공장을 봤다. 내일은 공장을 '설계'한다."

강사 노트

- Step 2에서 안 무너지는 학생은 "어떻게 지시했는지" 발표시키면 스펙의 필요성으로 자연 연결.
- 시뮬레이터 API 폴링 주기를 가이드에 고정 제시(과도 호출로 서버 부하 방지).

Day 2 — 설계: 스펙 주도 개발, 말로 만들되 엔지니어답게

학습목표

1. 프롬프트 단독 개발이 무너지는 원인을 컨텍스트 관점에서 설명할 수 있다.
2. Intent → PRD → OpenSpec → Given-When-Then의 산출물 체계를 이해한다.
3. 불완전한 스펙의 빈칸을 채우고 기존 항목의 정오를 판정할 수 있다.
4. 스펙으로부터 Supabase 기반 관제 백엔드를 생성하고 검증할 수 있다.

시간표 (240분)

시간	블록	방식
00:00-00:10	어제 회수: 무너진 이유 공개	강의
00:10-00:50	강의 2: 스펙 주도 개발(SDD)	강의
00:50-01:20	Lab 2-1: 빈칸 채우기 — 관제 스펙 완성	실습
01:20-01:30	휴식	
01:30-02:00	Lab 2-2: Supabase 가입 + 프로젝트 생성	실습
02:00-03:30	Lab 2-3: 스펙 → 구현 → 검증	실습
03:30-04:00	랩업: 어제 vs 오늘 대조 데모	발표

강의 2 슬라이드 목차 (약 18장) 1-3. 어제의 붕괴 해부: 컨텍스트 원도, 암묵 요구사항, 회귀 (regression) 4-6. 물리 세계에서 스펙이 더 절실한 이유: 소프트웨어 버그는 롤백하면 되지만 **설비**

를 잘못 세우면 라인이 선다 — 안전·비용이 걸린 시스템일수록 요구는 문서가 된다 7-11. 산출물 체계: System Intent(왜) → PRD(무엇을) → OpenSpec(정확히 무엇을) → Given-When-Then(어떻게 확인하나) 12-14. GWT 읽는 법 — 예제 3개를 전부 관제 도메인으로: ① 온도 임계 초과 시 알람 생성 ② 알람 등급별 담당자 배정 ③ 정비 작업지시 발행 조건 15-16. 빈칸 채우기 규칙: 채우기 / 정오 판정 / 근거 쓰기 17-18. "스펙을 쓰는 사람이 시스템을 소유한다" — 오늘 실습 예고

Lab 2-1: 빈칸 채우기 (30분) — 핸드온 가이드 §1

왜 하는가: AI가 코드를 짜는 시대에 엔지니어의 부가가치는 '정확한 요구 정의와 검증'으로 이동한다. 스펙을 읽고 판정하는 근육을 먼저 만든다.

- GitHub 배포 스펙 업데이트: **K-정밀 설비관제 시스템** — capability 3개: 설비·로봇 마스터 / 알람 규칙(임계·등급·에스컬레이션) / 정비 작업지시(발행→수행→완료 상태기계)
- 의도적 빈칸 6곳(Then 조건 누락)과 의도적 오류 3곳(잘못된 Then — 예: "온도 80°C 초과 시 알람 해제") 포함
- 개인 작업 20분 + 짝 교차검토 10분. 정답 공개는 Lab 2-3 검증에서 자연스럽게(스펙이 틀리면 구현이 틀리게 나옴).

Lab 2-2: Supabase 온보딩 (30분) — 핸드온 가이드 §2

- 회원가입 → 프로젝트 생성 → SQL 에디터에서 테이블 1개 수동 생성 → 대시보드 확인
- DB 첫 경험 기준 스크린샷 단계별 안내(오픈소스협회 교육에서 검증된 구성 재사용)

Lab 2-3: 스펙 → 구현 → 검증 (90분) — 핸드온 가이드 §3

- Step 1 (30분): 완성한 스펙을 Claude에 투입 → 테이블 스키마 + API + 관리 화면 생성. 어제 대시보드와 연결(어제 '보기만' 했던 공장에 알람·작업지시 데이터가 붙음). 시뮬레이터 스트림을 `sensor_readings`로 적재 시작(내일 자료 준비).
- Step 2 (30분): GWT 인수조건을 하나씩 실행하며 체크리스트 검증 — 시뮬레이터 강사 콘솔에서 온도 이벤트를 싸주면 학생 시스템에 알람이 뜨는지 확인. 빈칸/오류 판정이 틀렸던 학생은 버그로 확인 → 스펙 수정 → 재생성 (대조 해결).
- Step 3 (30분): 요구사항 카드 1장 추가 — 어제와 동일한 "알람 담당자 배정"을 이번엔 스펙 수정으로 처리 → 안 무너짐 (어제와의 직접 대조 = Day 2의 '와' 순간).

랩업 (30분): 어제 vs 오늘 나란히 비교 데모(지원자 2명). 자산화: 스펙 저장소 = 포트폴리오 강조 + 커밋. 예고: "공장을 보고, 설계했다. 내일은 공장의 이상을 '아는' 지능을 단다."

강사 노트

- 생성 대기시간 구간에 "AI가 뭘 하는지" 로그 읽는 법 안내 → 지루함을 학습으로 전환.
- Supabase 무료 티어의 동일 IP 대량 가입 정책 사전 테스트 필수.

Day 3 — 알기: 센서 데이터에서 이상을 읽는 에이전트

학습목표

1. 피지컬 AI에서 데이터가 오는 곳(센서·레거시 로그)과 수집·적재의 실무 비중을 설명할 수 있다.
2. 단순 RAG / Text2SQL / GraphRAG의 차이와 적용 기준을 구분할 수 있다.

- 3. 시계열 센서 데이터의 이상 구간 탐지 로직을 직접 구현할 수 있다.
- 4. 구현 기능을 MCP 도구로 노출하고, 에이전트가 이를 호출해 원인 추정·조치 제안(진단 리포트)까지 수행하게 할 수 있다.

시간표 (240분)

시간	블록	방식
00:00-00:55	강의 3: 현장 데이터와 판단 지능, 그리고 MCP	강의+영상
00:55-01:25	Lab 3-1: 레거시 센서 데이터 적재	실습
01:25-01:35	휴식	
01:35-02:25	Lab 3-2: 이상감지 알고리즘 직접 구현	실습
02:25-03:35	Lab 3-3: MCP 도구화 + 진단 에이전트	실습
03:35-04:00	랩업: 시연 + 본 과정 예고	발표

강의 3 슬라이드 목차 (약 20장) 1-4. 판단의 재료: 페루프의 '알기'는 데이터에서 시작 — 센서 실시간 스트림 + 레거시 데이터(수기 대장, 구형 설비 로그, CSV 덤프)가 현장의 90% 5-7. "데이터를 받아주는 기술"이 먼저다: 수집·정합·적재가 실무의 절반이라는 현실 8-12. 에이전트가 데이터에 접근하는 3가지 방식 대조(호서대 영상 재활용): 단순 RAG의 한계(집계 질문에 약함) / Text2SQL(정형 데이터의 정답) / GraphRAG(설비-부품-정비이력 관계 추론) — 각 방식이 답할 수 있는 질문과 없는 질문 13-16. MCP: 에이전트에게 '손'을 달아주는 표준 — 개념·구조, 오늘은 '읽는 손', 내일은 '움직이는 손' 17-18. 이상감지 미니 이론: 고정 임계 vs 이동평균 vs z-score — 오탐/미탐 트레이드오프 19-20. 오늘 실습 지도: "여러분이 짠 알고리즘을 AI가 도구로 쓴다"

Lab 3-1: 레거시 센서 데이터 적재 (30분) — 핸드온 가이드 §1

왜 하는가: 피지컬 AI의 첫 관문은 모델이 아니라 데이터다. 현장 CSV를 DB에 넣는 일이 실무의 절반이다.

- 배포 데이터: K-정밀 과거 7일치 CSV(부록 A — 이상 3곳 삽입: 온도 드리프트 / 진동 스파이크 / 센서 결측). 어제부터 쌓이는 실시간 스트림과 동일 스키마 → "과거+현재"가 한 테이블에 합쳐짐
- Claude로 적재 스크립트 생성 → 건수·기간 검증 쿼리 실행 → Day 2 설비 마스터와 FK 연결

Lab 3-2: 이상감지 알고리즘 직접 구현 (50분) — 핸드온 가이드 §2

왜 하는가: 이 랩만큼은 AI에게 통째로 맡기지 않는다. 전공에서 배운 알고리즘 감각이 AI 시대에 왜 여전히 무기인지 확인하는 시간이다.

- Step 1 (25분): 이동 윈도우 z-score 이상감지 함수를 직접 파이썬으로 작성(빠대 + TODO 3곳: 윈도우 통계 계산 / 임계 판정 / 결측 처리). C 배경용 인덱싱·슬라이싱 힌트 박스 제공.
- Step 2 (15분): 과거 데이터에 실행 → 심어둔 이상 3곳 중 몇 개를 잡는지 확인, 임계값 조정하며 정탐/오탐 체험.

- Step 3 (10분): 자기 구현 vs Claude 생성 구현 비교 — 결측 처리는 어느 쪽이 나은가 (대조 해결). "AI 결과물을 판정할 수 있는 사람"이라는 정체성 부여.

Lab 3-3: MCP 도구화 + 진단 에이전트 (70분) — 핸드온 가이드 §3

왜 하는가: 알고리즘이 혼자 돌면 스크립트, 에이전트가 도구로 쓰면 지능이 된다.

- Step 1 (25분): 이상감지 함수 + Supabase 조회를 MCP 도구 2개(`detect_anomaly`, `query_equipment`)로 노출(템플릿 제공, Claude로 완성)
- Step 2 (30분): 에이전트에 자연어 지시 — "지난 주 설비 이상을 점검하고, 이상이 있으면 해당 설비의 정비 이력을 조회해 원인 추정과 권고 조치를 담은 진단 리포트를 작성하라" → 도구 연쇄 호출로 리포트 생성 (Day 3의 '와' 순간)
- Step 3 (15분): 진단 리포트를 Day 1 대시보드에 표시 — 3일치가 한 화면에 합쳐짐. 단, 아직 에이전트는 '제안'만 한다 — "움직이는 건 내일"

랩업 (25분): 지원자 시연 + 강사 확장 데모(같은 구조가 실제 엣지 디바이스·로봇으로 이어지는 그림). 본 과정 예고 1장: "탐지의 다음 단계 — CNN으로 진동 파형을 읽고, PINN으로 물리 법칙을 심고, Edge AI로 설비 옆에서 돌린다. 그게 9월부터다."

강사 노트

- Lab 3-2가 컴공생 만족도가 가장 갈리는 지점 — 단계별 힌트 카드 3장, 15분 경과 시 힌트 1 일괄 공개.
- MCP 서버 로컬 실행 이슈(방화벽·포트) 사전 리허설, 실패 대비 강사 공용 MCP 서버 백업 경로.

Day 4 — 움직이기: 멀티에이전트 페루프와 팀 챌린지

학습목표

1. 싱글 에이전트의 한계(역할 과부하)와 책임 단위 분해 기준을 설명할 수 있다.
2. 감지→진단→조치의 멀티에이전트 페루프를 오케스트레이터-워커 구조로 구현할 수 있다.
3. 물리 세계에 명령을 내리는 시스템에서 HITL(사람 승인 게이트)이 왜 필수인지 설명할 수 있다.
4. 팀 단위로 페루프에 새 에이전트를 추가·통합하고 데모할 수 있다.

시간표 (240분)

시간	블록	방식
00:00-00:50	강의 4 + 라이브 시연: 페루프를 달다	강의+시연
00:50-01:00	챌린지 브리핑 + 팀 편성	안내
01:00-03:00	Lab 4-1: 팀 챌린지 (휴식 팀 자율)	팀 실습
03:00-03:50	데모 발표: 팀당 5분 × 8팀	발표
03:50-04:00	시상 + 본 과정 로드맵 공개	마무리

강의 4 슬라이드 목차 (약 16장) 1-3. 어제 에이전트의 부검: 지시 하나에 역할 4개 — 길어지는 프롬프트, 흔들리는 품질 4-7. 책임 단위 분해: 감지 에이전트 / 진단 에이전트 / 조치 에이전트 + 오케스트레이터 (Day 1의 Orchestrator-Worker 패턴 회수) 8-9. 물리 세계의 행동 도구: 시뮬레이터 제어 MCP 도구 공개 — `(set_equipment_speed)` / `(stop_equipment)` / `(dispatch_robot)` / `(ack_alarm)` 10-11. HITL: 물리 세계에선 되돌릴 수 없는 행동이 있다 — 감속은 자동, **정지·로봇 파견은 사람 승인** (승인 게이트 설계) 12-14. 라이브 시연: 강사 콘솔에서 EQ-03에 온도 드리프트 주입 → 감지 에이전트 포착 → 진단 에이전트 원인 추정 → 조치 에이전트가 감속 실행 + 정비 로봇 파견 승인 요청 → 승인 → 2D 공장 화면에서 AMR이 실제로 움직임 (특강 전체의 클라이맥스 예고) 15-16. 챌린지 규칙·평가 기준·팀 편성 안내

Lab 4-1: 팀 챌린지 (120분) — 핸드온 가이드 §1

왜 하는가: 4일의 마지막은 배운 것의 확인이 아니라, 스스로 페루프를 확장할 수 있다는 증명이다.

- 팀 편성: 4~5명 × 8팀 (사전 편성, 비컴공 2명은 분리 배치해 도메인·시나리오 역할 부여)
- 기본 미션(전 팀 공통, 60분 목표): 강사 시연과 동일한 감지→진단→조치 페루프를 자기 팀 네임스페이스에서 동작시키기 (템플릿 제공)
- 확장 미션(택1, 나머지 시간): 페루프에 새 에이전트 1개 추가
 - A. 예방정비 스케줄러 — 이상 빈도·가동시간 기반 정비 작업지시 자동 발행
 - B. AMR 작업 할당 — 정비·운반 요청 발생 시 배터리·거리 기준 최적 로봇 배차 (Routing 패턴)
 - C. 안전 인터록 — 이상 등급 '심각' 시 인접 설비 연쇄 감속 + 전체 승인 요청 (Parallelization)
 - D. 교대 리포트 — 교대 시점마다 지난 8시간 이상·조치 요약 리포트 자동 작성
 - 자유 주제 허용(강사 승인)
- 운영: 90분 개발 + 30분 데모 준비. 강사 1 + 스태프 순회, 팀당 '막힘 티켓' 2장(스태프 집중 지원권).

데모 발표 (50분): 팀당 5분(시연 3분 + 설계 설명 2분). 평가: 페루프 동작 여부 40 / 스펙 문서화 30 / 창의성 30. 즉석 학생 투표 병행.

시상 + 본 과정 로드맵 (10분)

- 시상: 최우수·창의·베스트스펙 3개 부문 (상품 예산 확인 필요)
- 로드맵 3장: ① 오늘까지: 시뮬레이터 위의 페루프(에이전틱 AI 파트) ② 본 과정: 센서/ML/CNN/PINN/Edge AI(수립E&E 파트)로 **진짜 물리 세계**의 페루프 + ProcessGPT 기반 프로세스 자동화 심화 ③ 참여기업 15개사·채용약정 4개사 — "이 포트폴리오를 들고 갈 곳이 정해져 있다"
- 마지막 멘트: "시뮬레이터의 공장을 움직였다. 9월부터는 진짜를 움직인다."

강사 노트

- ProcessGPT는 강사 데모로만 노출(딥에이전트 대기시간 리스크) — "본 과정에서 직접 다룬다"로 기대 전환.
- 안전망: 기본 미션 미완 팀은 60분 경과 시 '동작 보장 최소 경로' 카드 배포 — 전원이 페루프 데모에 성공하는 것이 최우선(확장 미션은 가점 개념).

- 제어 명령 충돌 방지: 네임스페이스 분리 최종 점검, 리허설 때 팀 동시 제어 시나리오 필수 테스트.

부록

A. 센서 데이터셋 사양 (레거시 CSV)

- 형식: CSV, CNC 6대 × 7일 × 1분 간격 (약 6만 행) — 시뮬레이터 실시간 스트림과 동일 스키마
- 컬럼: `equipment_id, timestamp, temperature, vibration, rpm, run_state`
- 삽입 이상 3종: ① EQ-03 온도 점진 상승(5일차, 드리프트형) ② EQ-05 진동 스파이크(3일차, 순간형) ③ EQ-01 센서 결측 2시간(6일차)
- 생성: 합성 스크립트(시드 고정, 재현 가능), 특강 1주 전 배포 리허설

B. 핸드온 가이드 제작 기준

- 일자당 30페이지 이내, 랩당 서두 반 페이지 "왜 하는가" 필수
- 스크린샷은 실습 환경 버전과 일치(특강 1주 전 최종 캡처)
- 제작 순서: 내용 먼저 완성 → 양식·로고는 마지막에 적용

C. 산출 문서·시스템 목록 (제작 필요)

산출물	분량/형태	비고
팩토리 시뮬레이터	웹 2D 뷰 + API + 강사 콘솔	3절 사양, 폴백 경로 병행
강의 슬라이드 4벌	24+18+20+16장	호서대 영상 자산 삽입
핸즈온 가이드 4벌	각 30p 이내	docx
스펙 깃데기 저장소	GitHub 1개	관제 도메인, 빈칸 6 + 오류 3
센서 데이터셋 + 생성 스크립트	CSV + py	부록 A
페루프 템플릿(Day 4 기본 미션)	저장소 내	오케스트레이터 빼대
사전 안내문(환경 설치)	2p	특강 1주 전 발송
요구사항 카드(Day 1)·확장 미션 카드(Day 4)	인쇄물	